

Configuración de Bases de Datos, Servidores y Software en el Equipo Cliente



Alejandro Sepúlveda Duarte

Lucy Estefany Izquierdo Jaramillo



Servicio Nacional de Aprendizaje — SENA

Centro del Comercio Regional Antioquia

Tecnología en Análisis y Desarrollo de Software

José Ignacio Botero Osorio

22 de julio de 2026

INFORME TÉCNICO DE CONFIGURACIÓN Y DESPLIEGUE DE SERVIDORES Y SOFTWARE EN EL EQUIPO DEL CLIENTE

SofInventory — Sistema ERP de Gestión de Inventarios y Venta



Campo	Información
Autores	Alejandro Sepúlveda Duarte / Lucy Estefany Izquierdo Jaramillo
Contacto	Alejandro: WhatsApp 3197751596 Lucy: WhatsApp 3145814384
Fecha	Junio 14 de 2026
Fecha de actualización	Julio 22 de 2026
Versión del documento	2.0 — Actualizado para despliegue con contenedores Docker
Tipo de documento	Informe técnico de instalación y despliegue
Proyecto	SofInventory – Sistema ERP de Gestión de Inventarios y Ventas
Dirigido a	Personal técnico o desarrolladores encargados del despliegue e implementación del sistema SofInventory

Contenido

Introducción	4
Objetivos	5
Objetivo general	5
Objetivos específicos	5
Descripción general del sistema SofInventory	6
Arquitectura tecnológica del sistema	6
Empaquetado y orquestación mediante Docker	7
Prerrequisitos — resumen de herramientas	8
Fase 1 — Instalación de prerrequisitos del software	8
Docker Desktop (Motor de contenedores)	8
Git y Git Bash (Control de Versiones y Terminal)	10
Fase 2 — Obtención del código fuente	13
Estructura del proyecto con Docker	14
Fase 3 — Configuración de las variables de entorno	15
Fase 4 - Despliegue con Docker Compose	17
Fase 5 — Protocolo de pruebas de aceptación	19
Verificación del estado de los tres contenedores	20
Prueba de acceso al sistema — login	20
Verificación de carga del dashboard	21
Solución de problemas comunes	23
Conclusiones	24
Glosario técnico	25
Referencias	28

Introducción

Durante el desarrollo del proyecto SofInventory se identificó la importancia de contar con un proceso documentado de despliegue que permitiera instalar y configurar el sistema en un equipo diferente al utilizado durante la etapa de desarrollo. Para lograrlo de manera eficiente, portable y reproducible, se adoptó Docker como plataforma de despliegue.

Docker permite empaquetar la aplicación junto con todas sus dependencias - base de datos PostgreSQL, servidor backend Django y servidor frontend Angular - en contenedores aislados que se ejecutan de forma consistente en cualquier sistema operativo que cuente con Docker instalado. Esto elimina la necesidad de instalar manualmente Python, Node.js, PostgreSQL o configurar entornos virtuales en cada equipo destino.

El presente informe técnico documenta el procedimiento realizado para el despliegue del sistema utilizando Docker Compose, que orquesta tres contenedores principales: una base de datos PostgreSQL, un servidor backend Django REST Framework y una aplicación frontend Angular servida mediante Nginx. Además, se incluyen consideraciones sobre variables de entorno, redes Docker, volúmenes de persistencia y validaciones funcionales.

La finalidad de esta guía es proporcionar un procedimiento organizado, reproducible y comprensible, permitiendo que otro desarrollador o encargado técnico pueda realizar el despliegue completo de SofInventory ejecutando un solo comando de Docker Compose.

Objetivos

Objetivo general

Documentar y estandarizar el proceso técnico de despliegue del sistema ERP SofInventory utilizando Docker y Docker Compose, asegurando la correcta orquestación de los tres contenedores (base de datos, backend y frontend) que componen la arquitectura de la aplicación, de manera portable y reproducible en cualquier entorno.

Objetivos específicos

- Establecer Docker Desktop como único prerequisite de infraestructura para el despliegue, eliminando la instalación individual de Python, Node.js, PostgreSQL y otras herramientas.
- Documentar la configuración de variables de entorno (.env) para la conexión entre contenedores y la parametrización segura del sistema.
- Instruir el despliegue del sistema completo mediante Docker Compose, incluyendo la construcción de imágenes multi-stage para frontend y backend.
- Describir la arquitectura de contenedores, incluyendo redes, volúmenes de persistencia, exposición de puertos y comunicación entre servicios.
- Definir los protocolos de prueba de aceptación inicial para verificar el correcto funcionamiento del sistema desplegado con Docker.
- Incluir guía de solución de problemas específica para errores comunes en entornos Docker.

Descripción general del sistema SofInventory

SofInventory es un sistema ERP orientado a la gestión de inventarios y procesos de venta empresarial. La plataforma permite administrar información relacionada con productos, usuarios, movimientos de inventario y operaciones comerciales mediante una arquitectura basada en tecnologías web modernas, desplegada utilizando contenedores Docker.

El sistema está compuesto por tres contenedores Docker que se comunican entre si a través de una red Docker interna: un contenedor de base de datos PostgreSQL, un contenedor backend con Django REST Framework y un contenedor frontend con Angular servido por Nginx. Docker Compose orquesta el levantamiento, construcción y configuración de todos los servicios.

Arquitectura tecnológica del sistema

SofInventory está diseñado bajo un enfoque de microservicios empaquetados en contenedores Docker, permitiendo un aislamiento completo entre los componentes y una ejecución consistente en cualquier sistema operativo.

Tabla 1

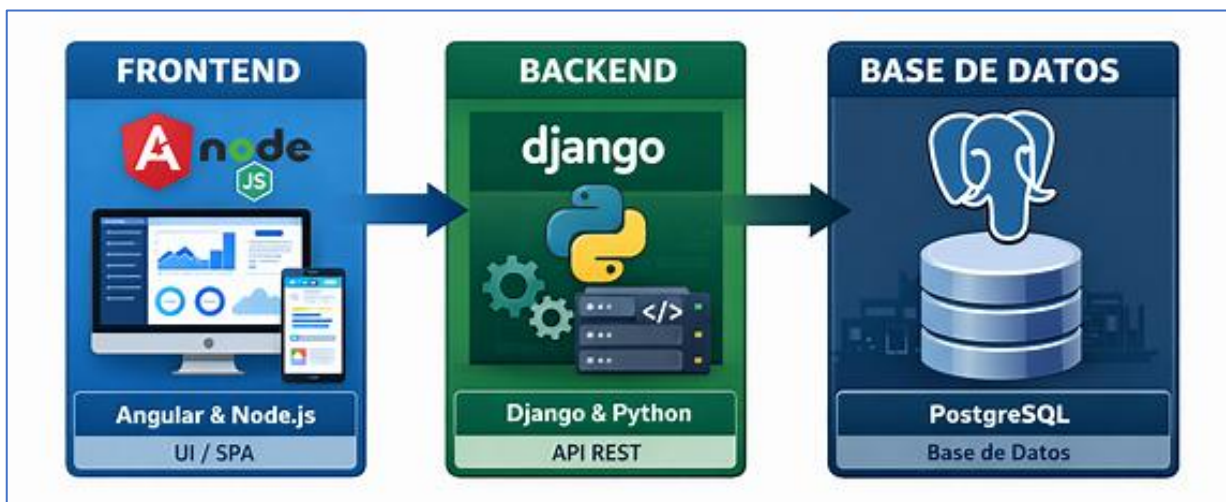
Arquitectura de los tres componentes del sistema

Capa	Tecnología	Puerto / acceso	Función
Frontend	Angular 21 / Node.js v22, servido en producción por Nginx	http://localhost/	Interfaz gráfica de usuario SPA desarrollada con TypeScript
Backend	Django 6 / Python 3.12, servido en producción por Gunicorn	http://localhost:8000/	Servicio API REST encargado de la lógica de negocio
Base de Datos	PostgreSQL 15 (imagen oficial postgres:15-alpine)	localhost:5432	Gestión y almacenamiento persistente de información

Nota. Elaboración propia. La tabla presenta los componentes tecnológicos principales utilizados en la arquitectura del sistema SofInventory y la función que cumple cada uno dentro de la solución.

Figura 1

Diagrama de arquitectura de SofInventory



Nota. Elaboración propia. Diagrama que representa la arquitectura del sistema.

La comunicación entre contenedores se realiza a través de la red Docker interna. El frontend envía solicitudes HTTP al backend mediante un proxy reverso configurado en Nginx, y el backend se comunica con PostgreSQL utilizando las variables de entorno inyectadas por Docker Compose. No es necesario exponer el puerto de base de datos al host, ya que la conexión ocurre dentro de la red Docker.

Empaquetado y orquestación mediante Docker

La arquitectura se distribuye en tres contenedores independientes —db, backend y frontend definidos en un archivo docker-compose.yml y orquestados con un único comando. Esto garantiza que el entorno de ejecución sea idéntico al utilizado durante el desarrollo, sin importar el sistema operativo del equipo destino.

Tabla 2

Empaquetado de los tres componentes del sistema en Docker

Contenedor	Imagen / build	Puerto expuesto	Persistencia
db	postgres:15-alpine (imagen oficial)	5432:5432	Volumen Docker sofinventory_postgres_data

backend	Build propio a partir de python:3.12-slim	8000:8000	Sin estado (depende de db)
frontend	Build propio (Node 20 para compilar Angular + Nginx para servir)	80:80	Sin estado

Nota. Elaboración propia. Contenedores definidos en docker-compose.yml para el despliegue local de SofInventory.

Prerrequisitos — resumen de herramientas

Para el despliegue de SofInventory con Docker, el único prerrequisito es la instalación de Docker Desktop. No es necesario instalar Python, Node.js, PostgreSQL, npm ni configurar entornos virtuales de manera manual, ya que todo se ejecuta dentro de los contenedores Docker.

Tabla 3

Herramientas y requisitos técnicos para el despliegue de SofInventory

Herramienta	Versión requerida	Comando de verificación	Propósito
Docker Desktop	Última versión estable (incluye Docker Engine y Docker Compose)	docker --version	Construcción y ejecución de los contenedores de db, backend y frontend
Git / Git Bash	Última versión estable	git --version	Obtención del código fuente del repositorio

Nota. Las versiones indicadas corresponden al entorno validado para el despliegue del sistema SofInventory. Se recomienda mantener Docker Desktop actualizado a su versión más reciente estable.

Fase 1 — Instalación de prerrequisitos del software

Con solo dos herramientas de infraestructura por instalar, esta fase se reduce a preparar Docker Desktop que incluye el motor de contenedores, Docker Compose y Git, necesarios para obtener el código fuente del proyecto.

Docker Desktop (Motor de contenedores)

Docker Desktop es la aplicación que permite construir, ejecutar y orquestar los contenedores de SofInventory en Windows, macOS o Linux. Incluye Docker Engine y Docker Compose en una sola instalación.

Informe técnico de configuración y despliegue del software en el equipo del cliente

Paso 1. Descargar Docker Desktop

Ingresa al sitio oficial: <https://www.docker.com/products/docker-desktop/>

Descargue el instalador correspondiente a su sistema operativo (Windows, macOS o Linux).

Paso 2. Ejecutar el instalador

Ejecute el archivo descargado y siga el asistente de instalación con las opciones predeterminadas.

En Windows, asegúrese de que la opción de utilizar el backend WSL 2 quede habilitada, ya que ofrece mejor rendimiento.

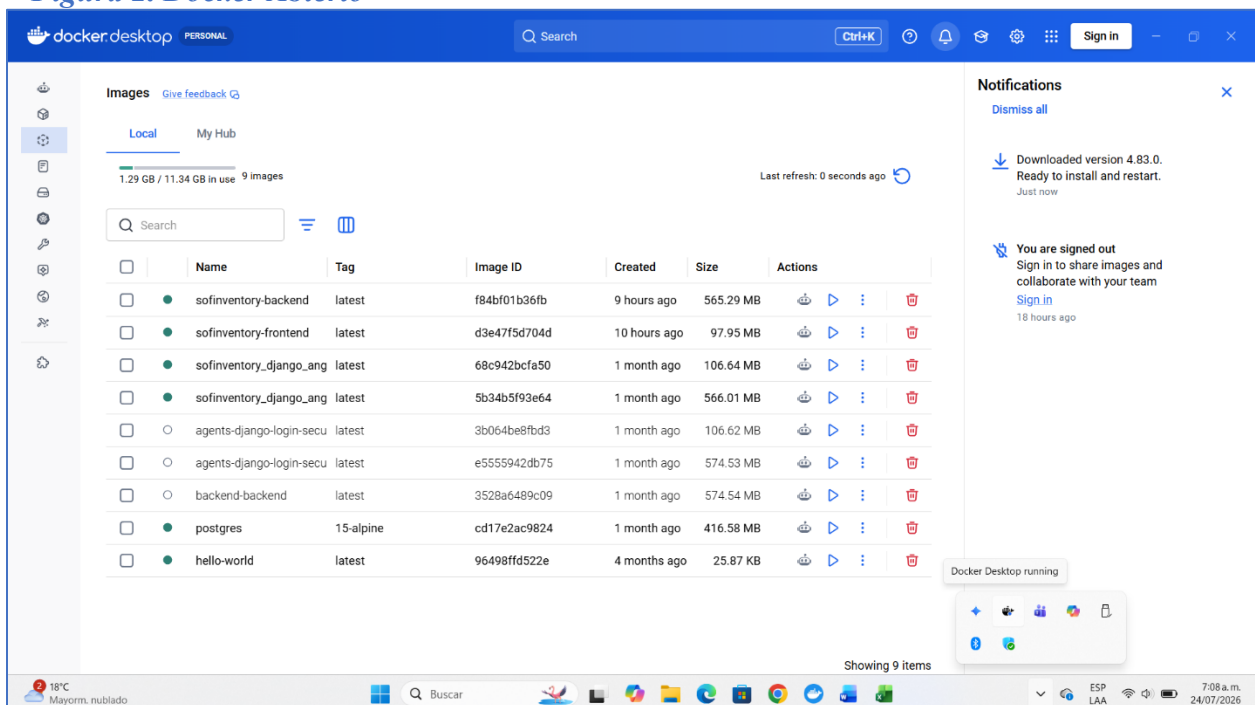
Paso 3. Reiniciar el equipo (si se solicita)

Algunos sistemas requieren reiniciar el equipo para completar la instalación de los componentes de virtualización necesarios para Docker.

Paso 4. Iniciar Docker Desktop y verificar el estado del motor

Abra Docker Desktop desde el menú de inicio y espere a que el ícono de la aplicación indique que el motor (Engine) está en ejecución ("Docker Desktop is running") antes de continuar.

Figura 2. Docker Abierto



Informe técnico de configuración y despliegue del software en el equipo del cliente

Paso 5. Verificar la instalación

Abra una terminal (PowerShell, CMD o Git Bash) y ejecute:

```
docker --version
```

```
docker compose version
```

Resultado esperado: la terminal debe mostrar el número de versión instalado tanto de Docker como de Docker Compose, sin mensajes de error. Ejemplo:

```
Alejo@DESKTOP-9PEFDP9 MINGW64 ~  
$ docker --version  
Docker version 29.5.3, build d1c06ef  
  
Alejo@DESKTOP-9PEFDP9 MINGW64 ~  
$ docker compose version  
Docker Compose version v5.1.4
```

Nota. Para complementar el procedimiento descrito en esta sección, se incluye el siguiente recurso audiovisual de apoyo para la instalación de **Docker**:

<https://www.youtube.com/watch?v=IMqYPAwX3iM>

Git y Git Bash (Control de Versiones y Terminal)

Git es una herramienta de control de versiones utilizada para obtener, administrar y actualizar el código fuente del proyecto SofInventory desde el repositorio correspondiente.

Git Bash es la terminal incluida con Git para Windows que permite ejecutar comandos con una sintaxis similar a los entornos Unix/Linux. Esta herramienta será utilizada durante el proceso de despliegue para ejecutar comandos relacionados con la descarga del proyecto, configuración de dependencias y administración del código fuente.

Paso 1. Descargar Git

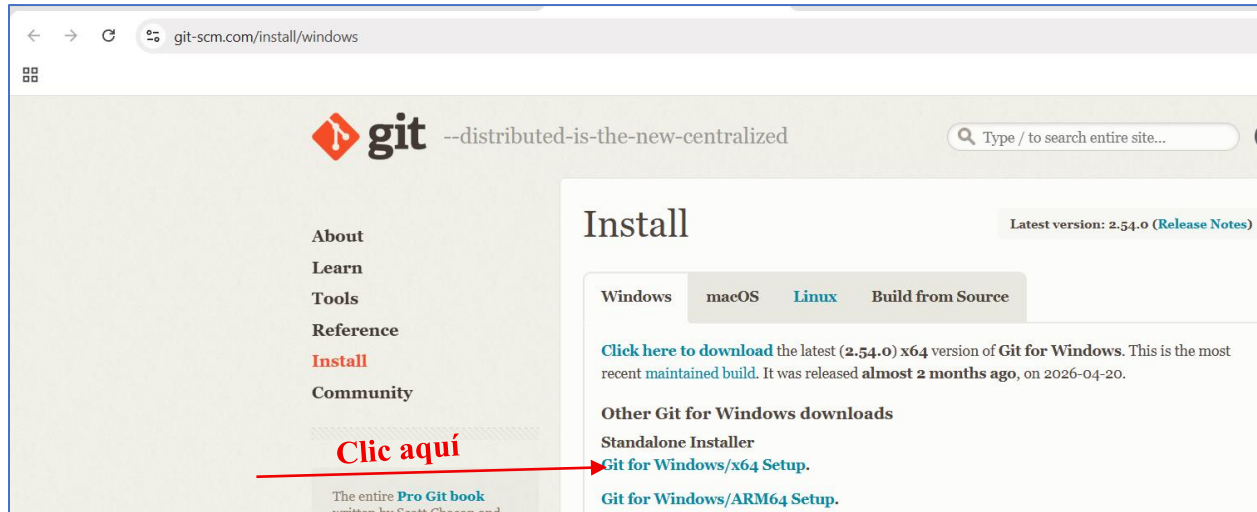
Ingresa al sitio oficial: <https://git-scm.com/download/win>

La descarga del instalador iniciará automáticamente para: **Windows 64 bits**

Descargue el instalador más reciente disponible.

Informe técnico de configuración y despliegue del software en el equipo del cliente

Figura 3. Página oficial de descarga de Git



Paso 2. Ejecutar el instalador

Ejecute el archivo descargado:

Git-XX.X.X-64-bit.exe

Si le aparece una ventana con un aviso

como: ¿Quieres permitir que esta

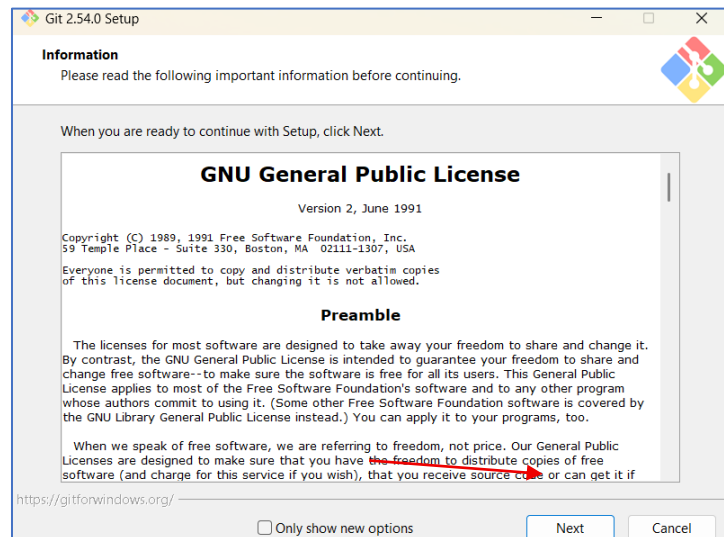
aplicación haga cambios en su

dispositivo?, clic en SI

Durante el asistente de instalación:

- Acepte los términos de licencia.
- Mantenga las opciones predeterminadas
- recomendadas.
- Presione: **Next → Next → hasta llegar a Install**

Figura 4. Asistente de instalación de Git



Las configuraciones por defecto son compatibles con el despliegue del proyecto SofInventory.

Informe técnico de configuración y despliegue del software en el equipo del cliente

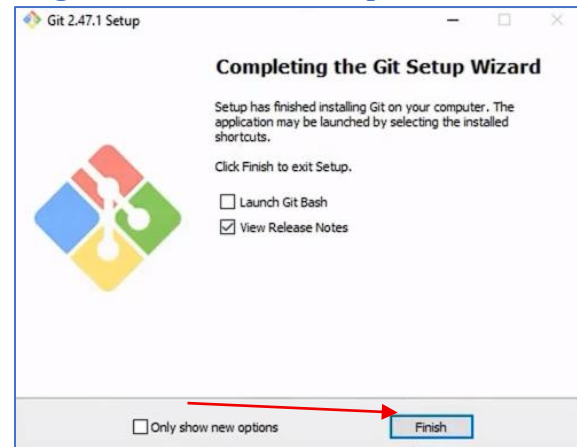
Paso 3. Finalizar instalación

Cuando finalice el proceso:

Presione: **Finish**

Git Bash quedará disponible desde el menú inicio de Windows.

Figura 5. Instalación completada



Paso 4. Verificar instalación

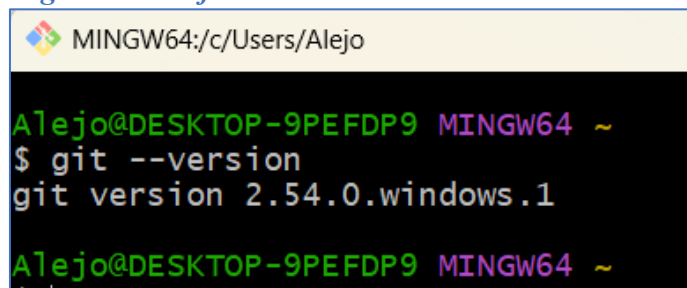
Abra: **Git Bash**

desde el menú inicio de Windows.

Ejecute el comando: `git --version`

Resultado esperado: `git version 2.x.x`

Figura 6. Verificación de Git



Si se muestra la versión instalada, Git quedó configurado correctamente.

Nota. Git será utilizado posteriormente para obtener el código fuente del sistema SofInventory y facilitar la administración de los archivos del proyecto durante el proceso de despliegue.

Nota. Para complementar el procedimiento descrito en esta sección, se incluye el siguiente recurso audiovisual de apoyo para la instalación de **Git**:

<https://www.youtube.com/watch?v=4sw0ROVaoQ0>

Fase 2 — Obtención del código fuente

Una vez instaladas las herramientas requeridas para el entorno de desarrollo, el siguiente paso consiste en obtener el código fuente del sistema SofInventory mediante Git.

El proceso de clonación permite descargar una copia completa del proyecto desde el repositorio configurado, manteniendo la estructura original de archivos y facilitando una instalación limpia del sistema.

Paso 1. Abrir Git Bash en el directorio de trabajo

Cree o seleccione la carpeta donde desea almacenar el proyecto.

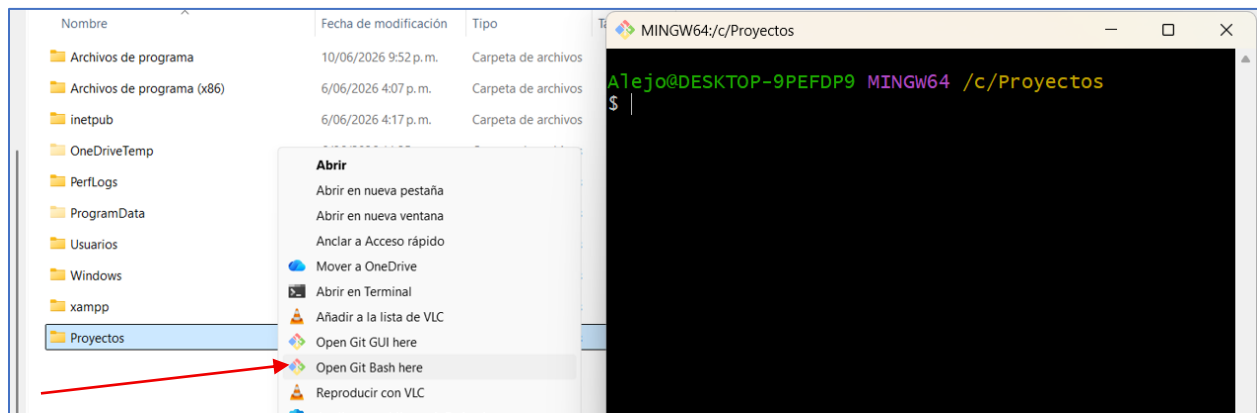
Ejemplo: C:\Proyectos

Desde el Explorador de archivos de Windows:

1. Ingrese a la carpeta seleccionada.
2. Haga clic derecho sobre un espacio vacío.
3. Seleccione: **Git Bash Here**

Se abrirá una terminal ubicada directamente en ese directorio.

Figura 7. Apertura de Git Bash en la carpeta del proyecto



Paso 2. Clonar el repositorio del proyecto

En la ventana de Git Bash ejecute el comando: `git clone URL_DEL_REPOSITORIO`

`git clone https://github.com/AlejandroSepulvedaDuarte/SofInventory.git`

Informe técnico de configuración y despliegue del software en el equipo del cliente

Este comando descargará todos los archivos necesarios del proyecto.

```
Alejo@DESKTOP-9PEFDP9 MINGW64 ~/Desktop/Proyectos
$ git clone https://github.com/AlejandroSepulvedaDuarte/SofInventory.git
```

Paso 3. Esperar la descarga del proyecto

Git iniciará la descarga de los archivos del repositorio. Durante el proceso se mostrará información del avance. Al finalizar aparecerá un mensaje similar a: **done**.

y se generará una nueva carpeta con el proyecto: **SofInventory**

```
Alejo@DESKTOP-9PEFDP9 MINGW64 ~/Desktop/Proyectos
$ git clone https://github.com/AlejandroSepulvedaDuarte/SofInventory.git
Cloning into 'SofInventory'...
remote: Enumerating objects: 271, done.
remote: Counting objects: 100% (271/271), done.
remote: Compressing objects: 100% (233/233), done.
remote: Total 271 (delta 30), reused 271 (delta 30), pack-reused 0 (from 0)
Receiving objects: 100% (271/271), 10.67 MiB | 12.11 MiB/s, done.
Resolving deltas: 100% (30/30), done.
```

Paso 4. Ingresar al directorio del proyecto

Ejecute el siguiente comando: `cd SofInventory`

Este comando permite ubicarse dentro de la carpeta principal del sistema.

```
Alejo@DESKTOP-9PEFDP9 MINGW64 ~/Desktop/Proyectos
$ cd SofInventory/

Alejo@DESKTOP-9PEFDP9 MINGW64 ~/Desktop/Proyectos/SofInventory (main)
$
```

Paso 5. Abrir el proyecto en Visual Studio Code

Desde la ubicación del proyecto ejecute: `code .`

Esto abrirá automáticamente todos los archivos del sistema en Visual Studio Code.

Estructura del proyecto con Docker

```
SofInventory/
  docker-compose.yml    # Orquestacion de servicios
  Dockerfile            # Build multi-stage (frontend + backend)
  .dockerignore         # Archivos excluidos del contexto Docker
  backend/
    Dockerfile          # Dockerfile especifico del backend
```

Informe técnico de configuración y despliegue del software en el equipo del cliente


```
docker-entrypoint.sh # Script de inicio (migraciones + gunicorn)
.env.example         # Plantilla de variables de entorno
.env                 # Variables de entorno (no se sube a Git)
requirements.txt
config/              # Configuración Django
frontend/
Dockerfile           # Dockerfile multi-stage del frontend
docker-entrypoint.sh # Inyección de BACKEND_URL en runtime
nginx.conf           # Configuración de Nginx proxy reverso
.env.example
src/
package.json
```

Nota. A diferencia de la versión 1.0, a partir de este punto ya no es necesario abrir por separado el backend y el frontend en un editor de código para instalar dependencias: el contenido descargado incluyendo el archivo docker-compose.yml en la raíz del proyecto es todo lo que se necesita para continuar con la siguiente fase.

Fase 3 — Configuración de las variables de entorno

Antes de ejecutar Docker Compose, es necesario configurar el archivo .env del backend con las credenciales de la base de datos y los parámetros de seguridad.

Paso 1. Crear el archivo .env del backend

Desde el Explorador de archivos o desde Visual Studio Code, cree un archivo llamado .env dentro de la carpeta backend/.

Importante: El archivo debe llamarse exactamente .env (sin extensión adicional).

Correcto: .env | **Incorrecto:** .env.txt

Paso 2. Configurar las variables

Dentro del archivo .env, agregue las siguientes variables:

```
SECRET_KEY=django-insecure-qnqdjpswh7=yoa*njm+j6b*cn-e(^@@l2a0*t-g1rz5^k57hnd
DB_NAME=db_backend_sofinventory
DB_USER=postgres
```

Informe técnico de configuración y despliegue del software en el equipo del cliente

```
DB_PASSWORD=ContraseñaDefinidaEnPostgreSQL
DB_HOST=db
DB_PORT=5432
```

```
DEBUG=0
ALLOWED_HOSTS=*
CORS_ALLOW_ALL_ORIGINS=True
```

Tabla 4.

Descripción de cada variable

Variable	Valor en Docker	Descripción
SECRET_KEY	Clave criptográfica	Clave utilizada por Django para funciones de seguridad
DB_NAME	db_backend_sofinventory	Nombre de la base de datos PostgreSQL
DB_USER	postgres	Usuario administrador de PostgreSQL
DB_PASSWORD	ContraseñaDefinidaEnPostgreSQL	Contraseña del usuario PostgreSQL
DB_HOST	db	Nombre del servicio en docker-compose.yml (no localhost)
DB_PORT	5432	Puerto de conexión PostgreSQL dentro de la red Docker
DEBUG	0	Modo de ejecución (0 = producción)
ALLOWED_HOSTS	*	Hosts permitidos por Django

Figura 8. Configuración del archivo .env

```
1 SECRET_KEY=django-insecure-qndjpswh7=yoa*njm+j6b*cn-e(^@l2a0*t-g1r25^k57hnd
2 DB_NAME=db_backend_sofinventory
3 DB_USER=postgres
4 DB_PASSWORD=alejo123
5 DB_HOST=db
6 DB_PORT=5432
7
8 DEBUG=0
9 ALLOWED_HOSTS=*
10 CORS_ALLOW_ALL_ORIGINS=True
11
```

Paso 3. Guardar la configuración

Guarde el archivo utilizando: **Ctrl + S**

Verifique que el archivo permanezca ubicado en: backend/.env

⚠ Advertencia: En el entorno Docker, DB_HOST debe ser "db" (el nombre del servicio en docker-compose.yml), NO "localhost". Los contenedores se comunican entre si usando los nombres de los servicios.

Paso 4. Variables del frontend

El frontend **NO** requiere un archivo .env manual para el despliegue con Docker, ya que el docker-entripoint.sh del frontend genera automáticamente el archivo env.js con la URL de la API en tiempo de ejecución usando la variable BACKEND_URL definida en docker-compose.yml.

Fase 4 - Despliegue con Docker Compose

Con el archivo .env configurado, el siguiente paso consiste en construir las imágenes Docker y levantar todos los servicios del sistema con Docker Compose.

Paso 1. Verificar que Docker Desktop este en ejecución

Verifique que el icono de Docker Desktop en la bandeja del sistema este verde (activo).

En la terminal digite este comando: `docker info`

Ejemplo:

```
PS C:\Users\Alejo\Desktop\Proyectos\SofInventory> docker info
Client:
Version: 29.5.3
Context: desktop-linux
Debug Mode: false
Plugins:
agent: Docker AI Agent Runner (Docker Inc.)
Version: v1.73.0
Path: C:\Program Files\Docker\cli-plugins\docker-agent.exe
```

Informe técnico de configuración y despliegue del software en el equipo del cliente

Si muestra información del daemon, Docker esta funcionando correctamente.

Paso 2. Levantar los servicios

Desde la terminal, ubicandose en la carpeta raiz del proyecto, ejecute:

```
docker compose up --build -d
```

Esperar hasta termine de la instalación completa. Ejemplo:

```
PS C:\Users\Alejo\Desktop\Proyectos\SofInventory> docker compose up --build -d

#33 DONE 0.0s
[+] up 5/5
    ✓ Image sofinventory-backend          Built
    ✓ Image sofinventory-frontend         Built
    ✓ Container sofinventory_final_backend Started
    ✓ Container sofinventory_final_frontend Started
    ✓ Container sofinventory_final_db      Started
What's next:
  Filter, search, and stream logs from all your Compose services in one place with
```

Este comando realiza las siguientes acciones:

- Construye la imagen del frontend: ejecuta el build multi-stage (compilacion Angular con Node.js y configuracion de Nginx).
- Construye la imagen del backend: instala dependencias de Python, copia el codigo Django y ejecuta collectstatic.
- Descarga la imagen de PostgreSQL 15 Alpine.
- Crea los contenedores y los conecta a una red Docker interna.
- Expone los puertos: 80 (frontend), 8000 (backend), 5432 (base de datos).
- Crea el volumen persistente sofinventory_postgres_data para la base de datos.
- Ejecuta los entrypoints que aplican migraciones y configuran servicios.

Paso 3. Verificar el estado de los contenedores

```
docker compose ps
```

Ejemplo:

```
PS C:\Users\Alejo\Desktop\Proyectos\SofInventory> docker compose ps
NAME                                IMAGE                                COMMAND
SERVICE    CREATED      STATUS      PORTS
sofinventory_final_backend  sofinventory-backend  "/docker-
entrypoint..." backend  2 minutes ago Up 2 minutes  0.0.0.0:8000-
>8000/tcp, [::]:8000->8000/tcp
sofinventory_final_db      postgres:15-alpine      "docker-
entrypoint.s..." db      2 minutes ago Up 2 minutes  0.0.0.0:5432-
>5432/tcp, [::]:5432->5432/tcp
sofinventory_final_frontend  sofinventory-frontend  "/docker-
entrypoint..." frontend  2 minutes ago Up 2 minutes  0.0.0.0:80-
>80/tcp, [::]:80->80/tcp
PS C:\Users\Alejo\Desktop\Proyectos\SofInventory>
```

Todos los contenedores deben mostrar el estado "Up" o "running". Si algún contenedor presenta errores, consulte la sección de solución de problemas.

Paso 4. Verificar los logs en la terminal

Para revisar los logs de todos los servicios: `docker compose logs -f`

Para revisar logs de un servicio específico:

`docker compose logs backend`

`docker compose logs frontend`

`docker compose logs db`

Fase 5 — Protocolo de pruebas de aceptación

Una vez que los tres contenedores principales del sistema se encuentren activos (db, backend y frontend), se debe ejecutar el siguiente protocolo de verificación para confirmar que el despliegue fue realizado correctamente y que SofInventory funciona de extremo a extremo. Estas pruebas validan la comunicación entre la base de datos PostgreSQL, el servidor backend Django REST Framework y la interfaz frontend Angular, orquestados mediante Docker Compose.

Informe técnico de configuración y despliegue del software en el equipo del cliente

Verificación del estado de los tres contenedores

Antes de realizar las pruebas funcionales, confirme que los tres contenedores se encuentren ejecutándose correctamente:

Tabla 4.

Estado de los tres componentes

Servicio	Cómo verificarlo	Indicador de éxito
Los tres contenedores	Ejecutar: docker compose ps	Los tres servicios (db, backend, frontend) en estado "Up" / "running"
Base de datos (db)	docker compose logs db	Mensaje "database system is ready to accept connections"
Backend	http://localhost:8000/api/	Respuesta de la API REST, sin error de conexión
Frontend	http://localhost/	Carga la interfaz de SofInventory

Nota. Elaboración propia

Prueba de acceso al sistema — login

Con los tres contenedores activos, realice la prueba de autenticación completa del sistema.

Tabla 5.

Descripción detallada para las pruebas de acceso

#	Acción	Descripción detallada
1	Abrir el sistema en el navegador	Abra Google Chrome o Microsoft Edge e ingrese la dirección: http://localhost/ (sin necesidad de especificar un puerto, ya que el contenedor frontend se sirve por Nginx en el puerto 80).
2	Verificar pantalla de inicio de sesión	Debe cargarse correctamente el formulario de autenticación de SofInventory.
3	Ingresar credenciales	Utilice el usuario administrador creado automáticamente por el contenedor backend a partir de las variables INITIAL_ADMIN_* del archivo .env.
4	Ejecutar inicio de sesión	Presione el botón "Entrar al sistema". El frontend enviará la solicitud de autenticación al backend Django a través del proxy configurado en Nginx.
5	Validar respuesta del sistema	Si las credenciales son correctas, el usuario será redirigido automáticamente al Dashboard principal.

Nota. Elaboración propia.

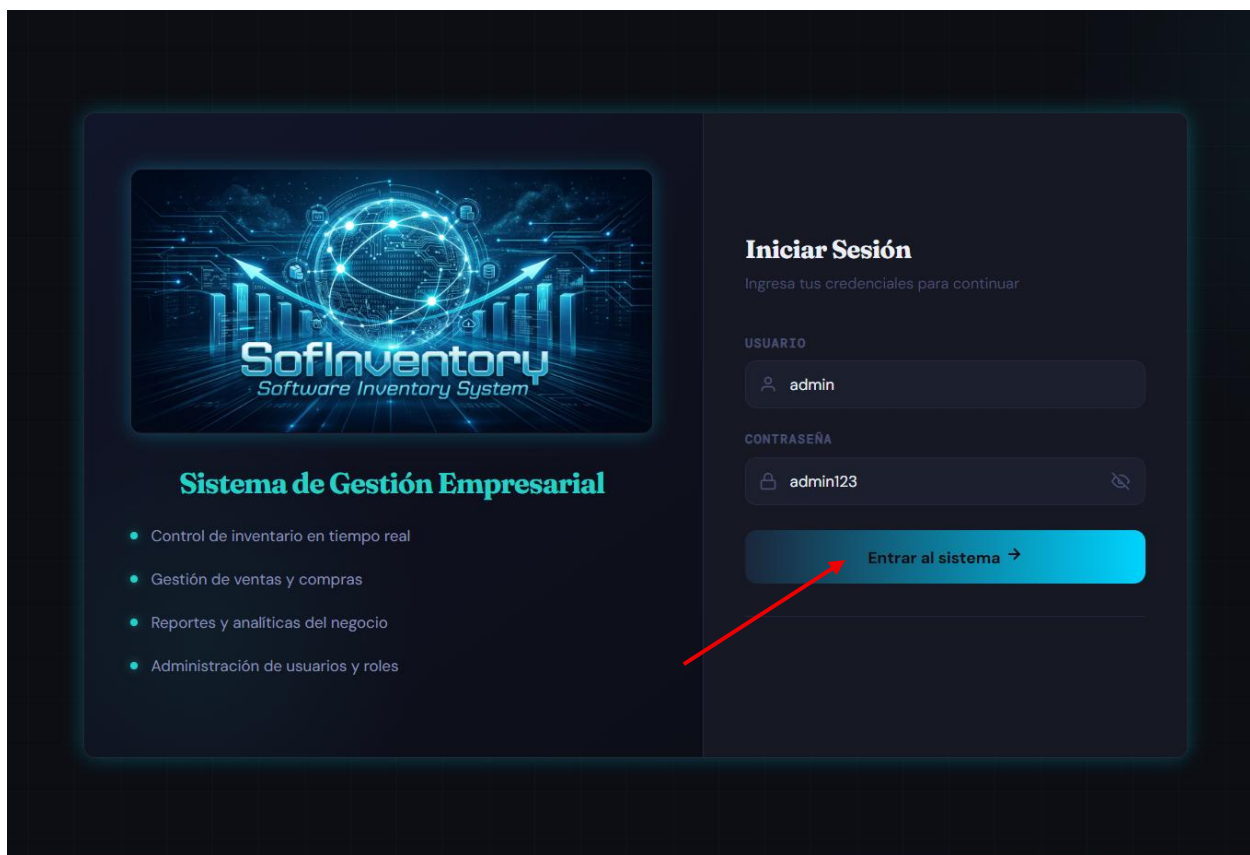
Tabla 6.

Credenciales Para poder ingresar al sistema

Campo	Valor por defecto	Función de seguridad
Username / Usuario	admin	Identificador único en la tabla usuarios de PostgreSQL
Password / Contraseña	admin123	Validado mediante hash PBKDF2-SHA256 en el servidor Django

Nota. Elaboración propia.

Figura 9. Pantalla de inicio de sesión



Nota. Elaboración propia.

Verificación de carga del dashboard

Después de ingresar correctamente al sistema, valide que la información principal sea cargada desde la API.

Informe técnico de configuración y despliegue del software en el equipo del cliente

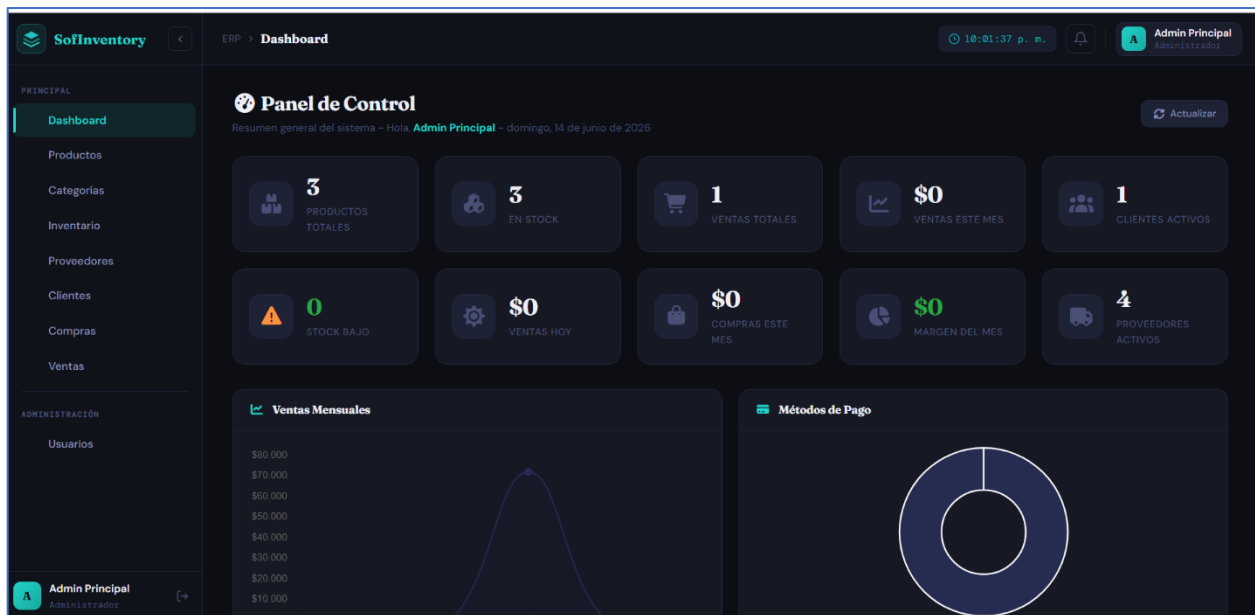
Tabla 7.

Resultados esperados al ingreso correcto

#	Acción	Resultado esperado
1	Visualizar Dashboard	El sistema debe mostrar la pantalla principal sin errores
2	Verificar datos cargados	Los módulos y elementos visuales deben mostrarse correctamente
3	Validar comunicación	La información debe provenir del backend Django conectado a PostgreSQL

Nota. Elaboración propia.

Figura 10. Dashboard principal del sistema



Nota. Si el sistema muestra errores durante el inicio de sesión o el Dashboard no carga correctamente, verifique primero que los contenedores estén activos con `docker compose ps` y revise los logs con `docker compose logs`.

Solución de problemas comunes

La siguiente tabla recoge los errores más frecuentes durante el proceso de despliegue y sus soluciones:

Tabla 8.

Posibles errores o causas durante el proceso de despliegue

Problema	Posible causa	Solución
El comando docker no se reconoce	Docker Desktop no está instalado o no se reinició la terminal tras instalarlo	Verifique la instalación de Docker Desktop y abra una nueva ventana de terminal
Los contenedores no arrancan / error de conexión a Docker	El motor de Docker Desktop (Engine) no está en ejecución	Abra Docker Desktop y espere a que indique "Engine running" antes de ejecutar docker compose up
El puerto ya está en uso (80, 8000 o 5432)	Otro proceso o servicio del equipo está usando ese puerto	Detenga el proceso que ocupa el puerto, o modifique el mapeo de puertos en docker-compose.yml (por ejemplo, "8080:80")
El contenedor backend se reinicia en bucle	Falta alguna variable obligatoria en backend/.env, o la base de datos aún no está lista	Revise el registro con docker compose logs backend e identifique la variable o el error reportado
Error al construir la imagen (build)	Conexión a internet inestable durante la descarga de dependencias, o falta de espacio en disco	Verifique la conexión a internet, ejecute docker system prune para liberar espacio y vuelva a intentar con docker compose up --build
Los cambios en el código no se reflejan	Las imágenes fueron construidas antes de modificar el código y no se han reconstruido	Vuelva a ejecutar docker compose up --build para reconstruir las imágenes afectadas
La página en localhost no carga	El contenedor frontend no terminó de iniciar o falló su compilación	Revise docker compose logs frontend; si hay errores de compilación, reconstruya con docker compose up --build frontend

Nota. Elaboración propia

Conclusiones

El desarrollo y despliegue del sistema ERP SofInventory mediante Docker permite establecer un proceso de instalación significativamente más simple, reproducible y portable que el despliegue manual de cada componente. Con un solo comando de Docker Compose, se levantan los tres contenedores (base de datos, backend y frontend) con todas sus dependencias configuradas automáticamente.

La adopción de Docker elimina la necesidad de instalar y configurar individualmente Python, Node.js, PostgreSQL, entornos virtuales y otras herramientas en cada equipo destino. Esto reduce drásticamente los posibles errores de compatibilidad entre versiones y facilita que cualquier desarrollador o encargado técnico pueda ejecutar el sistema sin conocimientos previos del entorno de desarrollo.

El uso de Dockerfiles multi-stage permite reducir el tamaño de las imágenes finales, ya que las herramientas de compilación (Node.js, npm) se descartan después de la fase de build y solo se conservan los archivos estáticos compilados servidos por Nginx en el frontend, y las dependencias mínimas de Python en el backend.

Glosario técnico

Para unificar conceptos y facilitar la lectura de este informe a analistas externos al proyecto, se definen los siguientes términos en orden alfabético. Respecto a la versión 1.0, se incorporan los términos propios de la contenerización con Docker.

API REST (Representational State Transfer): Interfaz de programación de aplicaciones que utiliza peticiones HTTP (GET, POST, PUT, DELETE) para transferir e intercambiar datos en formato JSON entre el frontend y el backend del sistema.

Backend: Componente del sistema que se ejecuta en el servidor, encargado de la lógica de negocio, la validación de datos, la seguridad y la comunicación con la base de datos. En SofInventory está desarrollado con Django REST Framework.

Contenedor: Instancia en ejecución de una imagen Docker, que aísla un proceso (por ejemplo, el backend o la base de datos) junto con todas sus dependencias, sin necesidad de instalarlas directamente en el sistema operativo anfitrión.

Dashboard: Panel de control visual que presenta de forma resumida los indicadores y datos más relevantes del sistema, como gráficos de stock, conteo de ítems críticos y KPI de ventas.

Docker Compose: Herramienta que permite definir y orquestar, mediante un único archivo (docker-compose.yml), varios contenedores relacionados —en este caso, la base de datos, el backend y el frontend— junto con sus puertos, variables de entorno y volúmenes.

Dockerfile: Archivo de texto que contiene las instrucciones necesarias para construir una imagen Docker, especificando la imagen base, las dependencias a instalar y el comando de arranque del contenedor.

Endpoint: Dirección o ruta específica de la API a través de la cual el frontend envía y recibe información del backend (por ejemplo, /api/auth/login/).

Frontend: Componente del sistema que se ejecuta en el navegador del usuario, encargado de la interfaz visual, la navegación y la interacción. En SofInventory está desarrollado con Angular.

Hash de Contraseña: Resultado alfanumérico unidireccional generado al procesar una contraseña mediante un algoritmo criptográfico (PBKDF2 con SHA256 en el caso de Django). Permite verificar contraseñas sin almacenarlas en texto plano.

Imagen (Docker): Plantilla inmutable que contiene el sistema de archivos, las dependencias y la configuración necesarias para ejecutar un contenedor. Se construye a partir de un Dockerfile.

KPI (Key Performance Indicator): Indicador clave de desempeño; métrica numérica utilizada para evaluar el estado del negocio, como ventas diarias o nivel de inventario.

Llave Foránea (Foreign Key): Restricción relacional en bases de datos que vincula un registro de una tabla con la llave primaria de otra tabla, garantizando la integridad referencial entre los datos.

Multietapa (Multi-stage build): Técnica de construcción de imágenes Docker en la que se utilizan varias etapas (por ejemplo, una para compilar Angular y otra para servirlo con Nginx), copiando únicamente los archivos resultantes de una etapa a la siguiente, reduciendo el tamaño final de la imagen.

Migración (Django): Archivo generado automáticamente por Django que describe los cambios en la estructura de la base de datos (creación o modificación de tablas) y permite aplicarlos de forma controlada. Con Docker, se ejecutan automáticamente al iniciar el contenedor backend.

Nginx: Servidor web utilizado en el contenedor frontend para servir los archivos ya compilados de Angular y actuar como proxy hacia el backend.

ORM (Object-Relational Mapping): Técnica que permite interactuar con la base de datos utilizando código Python en lugar de SQL directo. Django incluye su propio ORM para gestionar modelos y consultas.

Repositorio (Git): Espacio de almacenamiento donde se guarda el código fuente del proyecto junto con el historial completo de cambios, alojado en plataformas como GitHub.

Seed Data (Datos Semilla): Conjunto de datos iniciales o de prueba que se cargan en la base de datos para que el sistema sea funcional desde el primer arranque (roles, tipos de documento, usuario administrador). En la versión con Docker, este proceso se ejecuta automáticamente al iniciar el contenedor backend.

SPA (Single Page Application): Aplicación web que carga una sola página HTML y actualiza dinámicamente su contenido mediante JavaScript, sin recargar el navegador. Angular genera una SPA.

Superusuario: Usuario con privilegios administrativos totales dentro del sistema o de la base de datos, capaz de gestionar todos los módulos, roles y configuraciones.

Token de Acceso: Cadena de caracteres generada por el backend tras una autenticación exitosa, que el frontend envía en cada petición posterior para demostrar que el usuario tiene sesión activa.

Volumen (Docker): Mecanismo de almacenamiento persistente de Docker que permite conservar los datos de un contenedor (por ejemplo, los datos de la base de datos PostgreSQL) aunque el contenedor se elimine o se reconstruya.

Referencias

Django Software Foundation. (s.f.). Django 6.0 documentation. Recuperado de

<https://docs.djangoproject.com/>

PostgreSQL Global Development Group. (s.f.). PostgreSQL 15 Documentation. Recuperado de

<https://www.postgresql.org/docs/15/>

Nginx. (s.f.). Nginx documentation. Recuperado de <https://nginx.org/en/docs/>

Angular. (s.f.). Angular documentation. Recuperado de <https://angular.dev/>

Gunicorn. (s.f.). Gunicorn documentation. Recuperado de <https://docs.gunicorn.org/>

Locura Informática Digital. (2025, 6 agosto). *Cómo Descargar e INSTALAR GIT en Windows | FÁCIL y RÁPIDO (2025)* [Video].

YouTube. <https://www.youtube.com/watch?v=4sw0ROVaoQ0>